

异常处理

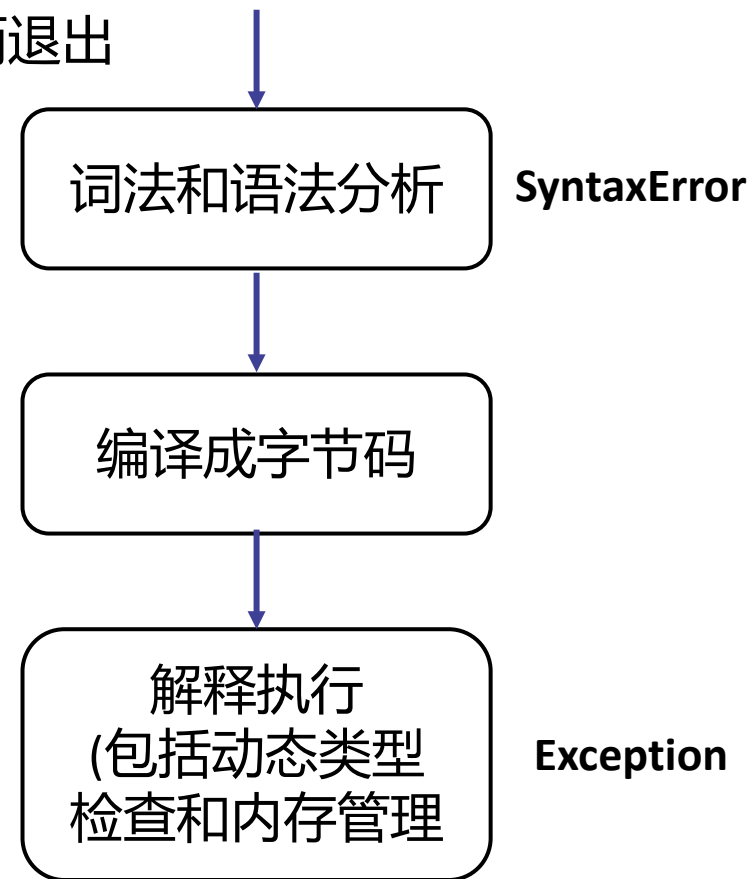
第13章

主要内容

- 常用异常类型
- 异常处理：try语句
- 异常处理：raise语句
- assert语句和traceback（了解）

错误和异常

- 语法错误：解释器在将代码转换为bytecode时发现不符合python语法时报SyntaxError
- 运行时错误(runtime error)：解释器在执行过程中检测到一个不合理的情况出现，即出现了异常(exception)
 - 导致异常出现的真正原因可能在之前的代码中
 - **如果不捕获异常**，解释器就会指出当前代码已无法继续执行下去而退出
- 逻辑错误(bug)：程序能够运行，但是逻辑上有错误
- 异常处理：
 - 可能出现哪些异常
 - 要捕获哪些异常，使得这些异常出现时程序可以继续执行



常见异常

```
>>> 10 / 0
Traceback (most recent call last):
  File "<pyshell#35>", line 1, in <module>
    10 / 0
ZeroDivisionError: division by zero

>>> namee + 1
Traceback (most recent call last):
  File "<pyshell#36>", line 1, in <module>
    namee + 1
NameError: name 'namee' is not defined
```

```
>>> '2' + 2
Traceback (most recent call last):
  File "<pyshell#37>", line 1, in <module>
    '2' + 2
TypeError: can only concatenate str (not "int") to str

>>> int('abc')
Traceback (most recent call last):
  File "<pyshell#38>", line 1, in <module>
    int('abc')
ValueError: invalid literal for int() with base 10: 'abc'
```

```
>>> s = 'abc'
>>> s[3]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range

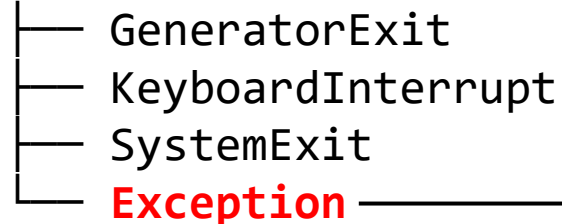
>>> open('NONEXIST')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileNotFoundError: [Errno 2] No such file or
directory: 'NONEXIST'
```

```
>>> s.abc
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'str' object has
no attribute 'abc'

>>> 4 +
      4 +
      ^
SyntaxError: invalid syntax
```

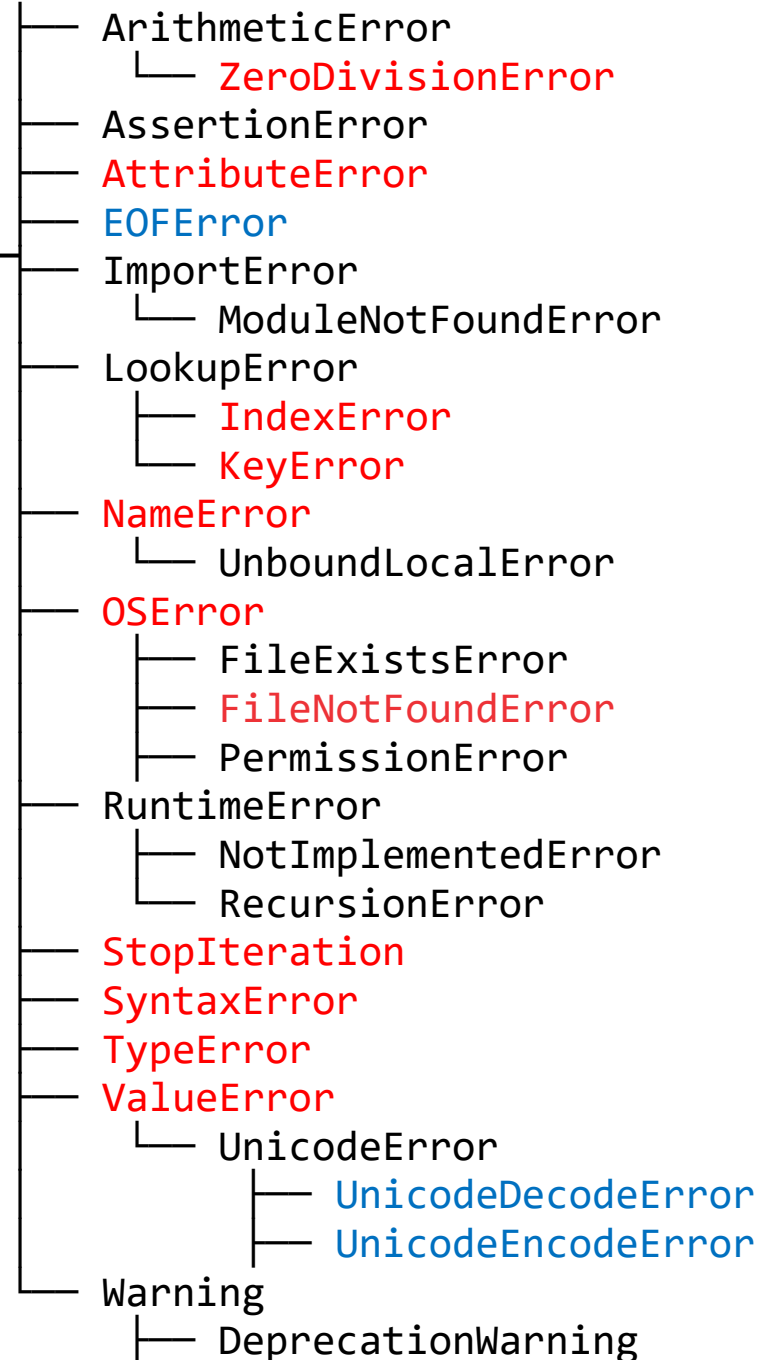
(主要的) 内置异常类型

BaseException



- `dir(__builtins__)` 查看内置函数以及内置的异常类型
- `KeyboardInterrupt`: 调用 `input()` 时用户输入 Ctrl-C 时抛出该异常
- `SystemExit`: 调用 `exit()` 或者 `sys.exit()` 退出程序时抛出该异常
- **Exception**: 用户自定义的异常应该继承 `Exception` 或其某个子类

```
>>> isinstance(ValueError, Exception)
True
>>> isinstance(Exception, BaseException)
True
```



异常对象

- 出现异常时构造一个异常对象
 - 某个具体的异常，其类型为某个异常类型
 - (来自于BaseException的实现) 属性args给出了创建该异常对象时传递的参数
 - BaseException: 还给出了__repr__和__str__的实现
- try语句用于捕获指定类型的异常
- 如果没有捕获最终导致解释器结束执行时，输出异常对象以及调用栈(traceback对象) 信息

```
>>> ValueError()
ValueError()
>>> ValueError('wrong')
ValueError('wrong')
>>> t = ValueError('wrong', 1)
>>> type(t)
<class 'ValueError'>
>>> t # print(repr(t))
ValueError('wrong', 1)
>>> print(t)
('wrong', 1)
>>> t.args
('wrong', 1)
```

```
try:
    print( 1 % 0)
except ZeroDivisionError as e:
    print(type(e), e)
    print(f'{e.__class__.__name__} = {e}')
    print(f'{e.args}')
```

```
<class 'ZeroDivisionError'> integer modulo by zero
e.__class__.__name__ = 'ZeroDivisionError'
e.args = ('integer modulo by zero',)
```

常用异常类型

异常	描述
Exception	几乎所有异常的基类(base class)
SyntaxError	语法错误
NameError	访问一个没有定义的变量
AttributeError	访问对象的属性时出错
IndexError	下标不存在
KeyError	字典的key不存在
TypeError	内置运算符或者某个函数作用的对象的类型不符
ValueError	内置运算符或某个函数作用的对象类型相符，但值不合适
ZeroDivisionError	除法类运算中除数为0
OSError	执行操作系统操作时出现错误，比如文件不存在等
AssertionError	断言异常
KeyboardInterrupt	在input()时键入<Ctrl-C>时
StopIteration	迭代器遍历时没有更多的元素
ModuleNotFoundError	import时相应的模块没有找到

异常处理: try语句

try:

<body>

except [expression [**as** identifier]]:
<exceptionBody>

...

except [expression [**as** identifier]]:
<exceptionBody>

else: # optional
<elseBody>

finally: # optional
<finallyBody>

- except关键字后面的expression应该为:
 - 异常类**, 如ValueError, TypeError等
 - 异常类的元组**, 表示其中任一异常出现, 如 (ValueError, TypeError)

- as identifier: identifier = 异常对象

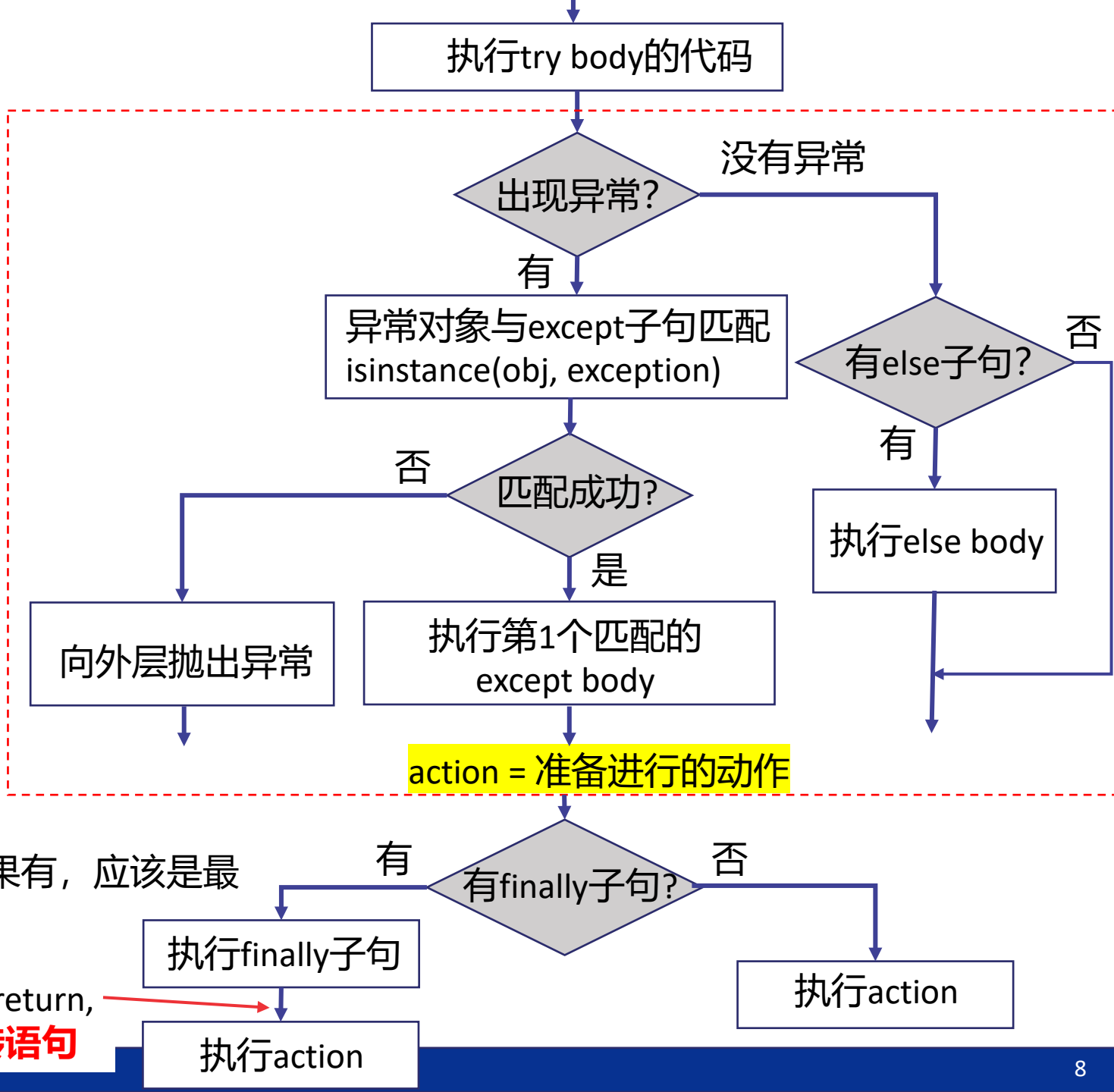
- except: 相当于 except BaseException, 如果有, 应该是最 后一个except子句

try:

<body>

finally:
<finallyBody>

如果finally没有执行到return, break, continue等**跳转语句**



示例1：输入整数

调用input()时：用户输入Ctrl-C，异常KeyboardInterrupt
用户输入Ctrl-D/Ctrl-Z：异常EOFError

例13-10, 13-11, 13-12变种

- 处理式策略：使用Python的异常处理机制
 - EAFP: Easier to Ask for Forgiveness than Permission
 - 允许错误发生, 在错误发生时进行处理.
 - 抓大放小：关注主要流程，较少发生的情形在except部分处理
 - 代码更易读写，也方便纠正错误
- 预防式策略：增加额外的逻辑(复杂性):
 - LBYL: Look Before You Leap
 - 要考虑用户输入
 - 要求为非空字符串，且全部为数字，或者第一个为符号，后面全为数字
 - 随着必须考虑的错误越来越多, 复杂性也随之增加→可能会掩盖程序的本来作用

```
def get_int():  
    while True:  
        try:  
            x = int(input("Please enter a number: "))  
            break  
        except ValueError as e:  
            print(type(e), e)  
    return x
```

```
def get_int2():  
    while True:  
        x = input("Please enter a number: ")  
        if not x:  
            continue  
        if x.isdigit() or (x[0] in '+-' and x[1:].isdigit()):  
            x = int(x)  
            break  
        print("That was not a valid number. Try again...")  
    return x
```

get_int.py

示例2：输入多个值+异常处理

- 循环结构之while循环，用户输入问题(Slide 30)：用户输入若干成绩，直到用户直接回车，不输入任何信息(空字符串) 时结束，求平均分。

average_score.py

```
def average_score2():
    total = n = 0
    text = input("请输入成绩[0, 100]: ")
    while text:
        try:
            score = float(text)
            if 0 <= score <= 100:
                total += score
                n += 1
            else:
                print(f"您输入的成绩{score}不在[0, 100]!")
        except ValueError as e:
            print('非法的浮点数', e)
            # t = e

        text = input("请输入成绩[0, 100]: ")

    if n > 0:
        print('平均成绩 = %.2f' % (total / n))
```

- expression as instance : instance 为异常对象
- 在exception block引入的变量在执行完该block仍然可用，但 instance 仅在该block中可用

```
except Exception as e:
```

类似于:

```
try:
```

```
    pass # except body
```

```
finally:
```

```
    del e
```

示例3: 多个except子句+else

- 在异常出现时, 按照顺序匹配异常, 一旦成功, 则执行对应的except body, 后续except块不再匹配执行
 - 异常类型有层次关系时, 子类应该先于大类
 - except:** 等价于except BaseException:, 可捕获所有异常(不建议)
 - KeyboardInterrupt, SystemExit等异常一般不建议捕获
 - except Exception as e** 捕获绝大部分异常
- else子句是可选的, 在没有异常出现时执行
 - 对应的代码也可放在try语句中, 但要注意两者的区别
 - try语句中的异常可被当前try语句捕获
- except子句和else子句中的代码在执行过程中也可能出现异常
 - 这些异常并不会被当前的try语句捕获
 - 如果对应的子句没有捕获该异常, 该异常将被抛出

division.py

```
def division():
    try:
        x = eval(input("请输入被除数: "))
        y = eval(input("请输入除数: "))
        z = x / y
    except ZeroDivisionError as e:
        print(type(e), e)
        print("除数不能为零")
    except TypeError as e:
        print(type(e), e)
        print("被除数和除数应为数值类型")
    except NameError as e:
        print(type(e), e)
        print("变量不存在")
    except Exception as e:
        print(type(e), e)
    else:
        print(f"{x} / {y} = {z}")
    finally:
        print("finally: clean up code here")

    print("after try statement.")
```

示例3：多个except子句+else

- 当有多个except块而且处理相同时，可以使用**元组的形式**，表示出现的异常匹配元组中的**任意一种异常**时，执行相应的except语句块

division.py

```
def division2():  
    try:  
        x = eval(input("请输入被除数: "))  
        y = eval(input("请输入除数: "))  
        z = x / y  
    except (ZeroDivisionError, TypeError) as e:  
        print(type(e), e)  
        print("被除数和除数应为数值类型,且除数不能为零")  
    except NameError as e:  
        print(type(e), e)  
        print("变量不存在")  
    except Exception as e:  
        # except 等价于 except BaseException  
        print(type(e), e)  
    else:  
        print(f"{x} / {y} = {z}")  
    finally:  
        print("finally: clean up code here")  
  
    print("after try statement.")
```

finally子句

- 不管有没有异常，异常是否捕获都会执行finallyBody：
 - 可进行清理工作，以便释放资源
 - 关闭文件、数据库和网络连接断开、上下文处理等
- try语句也可仅包含finally，表示不捕获任何异常，不管异常是否出现都会执行finallyBody

```
try:
    print('....')
    main()
finally:
    print('good luck!')
```

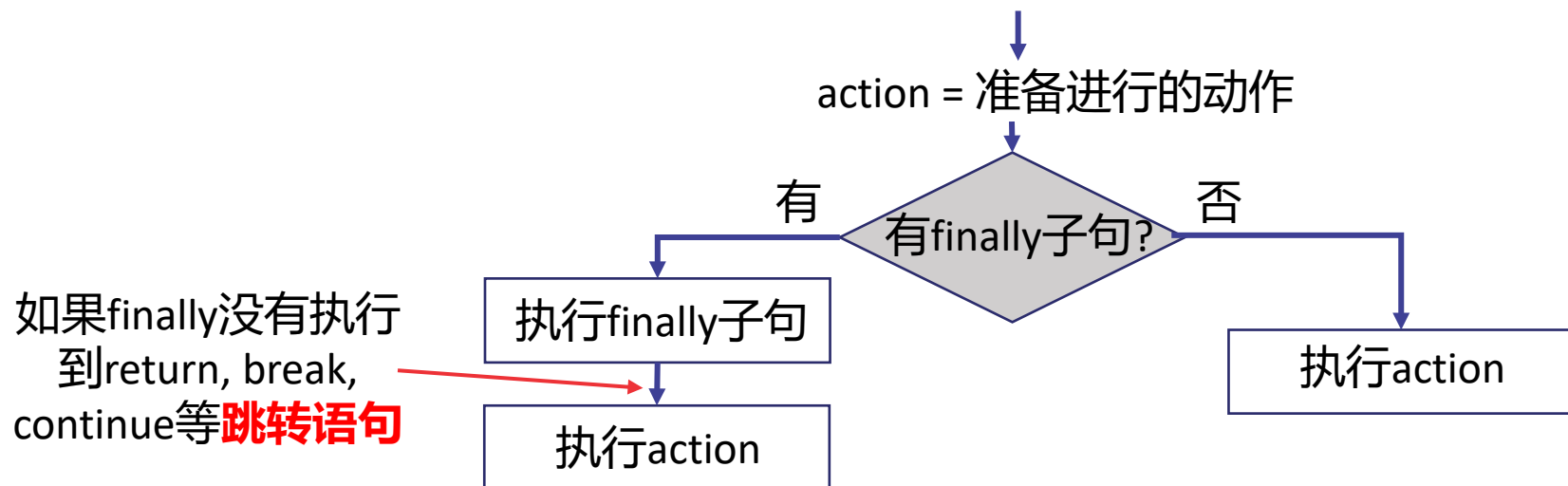
```
try:
    <body>
except [expression [as identifier]]:
    <exceptionBody>

...
except [expression [as identifier]]:
    <exceptionBody>
else: # optional
    <elseBody>
finally: # optional
    <finallyBody>
```

```
try:
    <body>
finally:
    <finallyBody>
```

finally子句

- finally一定会执行，如果finally块执行到break, continue, return时，之前本来要执行的动作不再执行



```
while True:
    try:
        1/0
    finally:
        break
```

右边的三个函数调用时都返回'finally'，原来的动作被取消

- 没有出现异常
- 出现了异常且未捕获
- 捕获了异常，但是在except子句中又出现了异常

```
def bar():
    try:
        return 'try'
    finally:
        return 'finally'

def foo():
    try:
        1 / 0
    finally:
        return 'finally'

def foobar():
    try:
        1 / 0
    except:
        raise ValueError
    finally:
        return 'finally'
```

finally_demo.py

raise语句：主动抛出异常

raise [expression [from orig_expression]]

• expression必须是：

- 一个异常类的实例对象，比如Exception('foo occurred!')
- 一个异常类，比如Exception。首先创建一个该类的实例对象(参数为空)，然后抛出该异常对象

```
raise Exception('foo occurred!')
raise Exception      # raise Exception()
```

```
def compute_circle_area(radius):
    if not isinstance(radius, (float, int)) or radius < 0:
        raise ValueError('radius should be a number larger than 0')
    return math.pi * radius * radius
```

circle_area.py

13-13采用assert
13-18采用raise
更加建议后者

```
raise [expression [from orig_expression]]
```

- **单独的raise一般出现在except块中**，表示重新抛出正在匹配的异常。如果当前没有异常时进行raise，则抛出RuntimeError异常(No active exception to reraise)

```
import logging
logging.basicConfig(filename='main.log',
                    format='%(asctime)s [%(levelname)s] %(message)s',
                    level=logging.DEBUG)
logging.info('logging started.')
def get_threshold(prompt=None) :
    try:
        thresh = int(input(prompt))
        return thresh
    except ValueError as e:
        # print(type(e), e)
        logging.error('get_threshold failed')
        logging.exception(e)
        raise
```

```
>>> logging.warning('This is a warning message!')
WARNING:root:This is a warning message!
```

get_threshold.py

日志模块：logging

- 记录**哪些级别**(缺省WARNING)之上的消息
- DEBUG<INFO<WARNING<ERROR<FATAL
- 记录在**哪个文件**，缺省标准输出
- **日志格式如何**，缺省'%(levelname)s: %(name)s: %(message)s'

主要方法：

- debug(msg)
- info(msg)
- warning(msg)
- error(msg)
- fatal(msg)
- exception(e)

raise语句

在except块处理已有的异常时(假设为context_exception):

```
raise expression from cause_expression
```

```
raise expression
```

```
raise expression from None
```

输出信息的不同

if `exc.__cause__`:

输出显示记录的关联异常

The above exception was the direct cause of the following exception

elif `exc.__context__` and not `exc.__suppress_context__`:

输出隐式记录的上下文异常

During handling of the above exception, another exception occurred:

- 给出了将两个异常连接起来的方法, 表示现在抛出的异常expression是由于另一个异常orig_expression引起的
 - (隐式)记录上下文异常: `exception.__context__ = context_exception`
 - (显式)记录关联的异常 (`raise expression from cause_expression`): `expression.__cause__ = cause_expression`; `exception1.__suppress_context__ = True`
 - `raise exception1 from None`: 显式记录关联异常为None, 不会显示 (隐式) 记录的上下文异常
 - 显式记录的异常优先

```
raise Exception('foo occurred!') from Exception('an error occurred!')
```

Exception: an error occurred!

The above exception was the direct cause of the following exception:

Traceback (most recent call last):

File "<pyshell#13>", line 1, in <module>

raise Exception('foo occurred!') from Exception('an error occurred!')

Exception: foo occurred!

raise from 示例

```
try:
    a = 1 / 0
except ZeroDivisionError as e:
    raise
```

```
Traceback (most recent call last):
...
a = 1 / 0
      ~~~~
ZeroDivisionError: division by zero
```

raise_from.py

```
try:
    a = 1 / 0
except ZeroDivisionError as e:
    raise ValueError('cannot divide by zero')
```

```
Traceback (most recent call last):
...
a = 1 / 0
      ~~~~
ZeroDivisionError: division by zero
```

During handling of the above exception, another exception occurred:

```
Traceback (most recent call last):
...
raise ValueError('cannot divide by zero')
ValueError: cannot divide by zero
```

```
try:
    a = 1 / 0
except ZeroDivisionError as e:
    raise ValueError('cannot divide by zero') from e
```

```
Traceback (most recent call last):
...
a = 1 / 0
      ~~~~
ZeroDivisionError: division by zero
```

The above exception was the direct cause of the following exception:

```
Traceback (most recent call last):
...
raise ValueError('cannot divide by zero') from e
ValueError: cannot divide by zero
```

```
try:
    a = 1 / 0
except ZeroDivisionError as e:
    raise ValueError('cannot divide by zero') from None
```

```
Traceback (most recent call last):
...
raise ValueError('cannot divide by zero') from None
ValueError: cannot divide by zero
```

自定义异常

class语句定义一个新的类型，该类型继承Exception类

- 类型名建议采用驼峰命名法，每个单词首字母大写
- Class Body部分给出了该类型的所有对象的模板(blueprint)
- 继承：子类继承了父类的模板，允许子类添加新的方法或者覆盖父类的方法

```
class TypeName:  
    # 等价于class TypeName(object):  
        <Class Body>  
  
class TypeName(ParentType):  
    <Class Body>
```

average_score2.py

```
class ScoreValueError(Exception):  
    pass  
  
def average_score(scores):  
    if not scores:  
        raise ScoreValueError("No scores provided.")  
    total, count = 0, 0  
    for score in scores:  
        if not isinstance(score, (int, float)):  
            raise ScoreValueError("Invalid score type.")  
        if score < 0 or score > 100:  
            raise ScoreValueError("Invalid score range.")  
        total += score  
        count += 1  
    return round(total / count, 2)
```

assert语句

- 断言语句: `assert expression[, reason]`
- 对于必须满足的条件expression进行验证
- 要求expression真值判断的结果为真, 这样才会顺序执行assert语句之后的语句
 - 如果表达式expression判断为假, 则**抛出异常AssertionError**, 可选的reason给出了原因

```
def assert_divzero():  
    a = int(input('请输入被除数:'))  
    b = int(input('请输入除数:'))  
    assert b != 0, '除数不能为0!'  
    c = a / b  
    print(f"{a} / {b} = {c}")
```

assert_divzero.py

```
>>> assert_divzero()  
请输入被除数:15  
请输入除数:0  
Traceback (most recent call last):  
  assert_divzero(), line 190, in assert_divzero  
    assert b!=0, '除数不能为0!'  
AssertionError: 除数不能为0!
```

```
if __debug__:  
    if not expression:  
        raise AssertionError  
        # raise AssertionError(reason)
```

什么时候__debug__为True?

- 目前的实现中, 正常情况下, __debug__为True
- 当Python脚本以-O选项执行时, __debug__为False, assert语句不会产生任何代码, 相当于pass语句

```
>>> b = 0  
>>> assert(b != 0)    # assert b != 0  
Traceback (most recent call last):  
AssertionError  
>>> assert(b != 0, 'b不能等于0')  
<stdin>:1: SyntaxWarning: assertion is always  
true, perhaps remove parentheses?
```

回溯traceback

拓展

- 发生异常退出时，输出异常和traceback信息
- 如果已经捕获异常，要输出调用栈信息？ traceback模块

```
import traceback
def traceback_demo():
    try:
        C()
    except:
        traceback.print_exc()
```

traceback_demo.py

方法	描述
sys.exc_info()	返回当前异常信息(type, value, traceback)
traceback.print_exc()	输出当前异常信息（包括traceback信息）
traceback.format_exc()	返回当前异常信息（包括traceback信息）字符串

```
>>> traceback_demo()
Traceback (most recent call last):
  File "C:\Users\dlmao\Documents\src\PythonClass\chap8\traceback_demo.py", line 19, in traceback_demo
    C()
  File "C:\Users\dlmao\Documents\src\PythonClass\chap8\traceback_demo.py", line 14, in C
    B()
  File "C:\Users\dlmao\Documents\src\PythonClass\chap8\traceback_demo.py", line 11, in B
    A()
  File "C:\Users\dlmao\Documents\src\PythonClass\chap8\traceback_demo.py", line 8, in A
    1/0
ZeroDivisionError: division by zero
```

```
def A():
    1/0

def B():
    A()

def C():
    B()
```

traceback_demo

C

B

A